

SMART INDIA HACKATHON 2026

Problem Statement 26191

Intelligent Identification of Hazard-Based Red Zones, Carrying Capacity Assessment, and Immediate Relocation Needs for Vulnerable Habitations

FINAL MASTER BLUEPRINT + AI DEVELOPMENT PLAYBOOK

Version 1.0 — Prototype scope frozen

Frozen prototype geography: Chamoli District, Uttarakhand

Core hazards: Landslide + Flood/Flash Flood + Extreme/Heavy Rainfall as a triggering input

Purpose: This document is the single source of truth for implementation. Every coding AI must follow this architecture, schemas, formulas, constraints, naming conventions, and acceptance criteria unless this document is deliberately versioned.

Prepared as an implementation specification for layer-by-layer development with coding AI.

1. Executive Decision — What We Are Building

The prototype is a GIS-enabled decision-support system for proactive relocation planning in Chamoli district. It is not merely a disaster map, a landslide predictor, or a population database. The system combines hazard, exposure, vulnerability, disaster history, site suitability, carrying-capacity constraints, relocation priority, and allocation into one auditable workflow.

The final product should answer five questions:

- Which parts of Chamoli are unsuitable for permanent habitation under the prototype's multi-hazard rules?
- Which vulnerable habitations are exposed to those hazards and how severe is their relocation need?
- Which candidate sites are comparatively safer and operationally suitable for relocation?
- How many people can each candidate site realistically support under modeled constraints?
- Which habitation should be addressed first, to which site, and why?

Non-negotiable product principles

- GIS/deterministic rules, machine learning, and optimization remain separate modules.
- Every important score has an explanation and a data-provenance trail.
- Prototype thresholds are configurable and explicitly labeled as prototype assumptions unless supported by an authoritative standard.
- No claim of government certification, legal red-zone declaration, or guaranteed prediction.
- No invented datasets, APIs, credentials, or government integrations.
- No 'real-time' label unless an actual update mechanism and source are implemented.
- The first release favors a defensible complete pipeline over a broad but shallow India-wide system.

Decision status: OD-01 through OD-12 are now frozen for the prototype. Any later change requires a version bump and a written impact note.

2. Frozen Open Decisions (OD-01 to OD-12)

ID	Frozen decision	Prototype choice	Reason / implementation note
OD-01	Prototype geography	Chamoli District, Uttarakhand	Strong alignment with landslide/flash-flood context and available government GIS/disaster resources.
OD-02	Initial hazard set	Landslide; Flood/Flash Flood; Extreme/Heavy Rainfall trigger	Keeps the first release scientifically and engineering-wise defensible.
OD-03	Primary data policy	Authoritative/open government sources first; documented open alternatives second	No fabricated or unverifiable inputs.
OD-04	Spatial/temporal resolution	Vector boundaries at native resolution; canonical analysis grid 30 m; retain native source resolution; daily rainfall + available hourly CWC telemetry for update-capable modules	30 m is a practical prototype analysis grid; do not upsample coarse data and call it higher resolution.
OD-05	Red Zone rule	Hybrid hard constraints + normalized multi-hazard risk; configurable thresholds	Red Zone is a prototype decision layer, not a legal notification.
OD-06	Vulnerability weights	Population exposure 35%; social vulnerability proxies 25%; critical-service access 20%; infrastructure dependency 10%; historical disaster recurrence 10%	Weights are prototype assumptions stored in configuration.
OD-07	Site suitability weights	Hazard safety 30%; slope 15%; road access 15%; water 15%; healthcare 10%; education 5%; land-use 5%; service proximity 5%	Hard exclusions are applied before scoring.
OD-08	Carrying capacity	Bottleneck-based effective capacity with physical, water, sanitation, access, health and safety constraints	Output is modeled effective capacity, not an engineering certification.
OD-09	Relocation priority	Risk 40%; exposed population 25%; vulnerability 20%; response/access difficulty 10%; recurrence 5%; tiers: Immediate ≥ 0.75 , Short $0.55 < 0.75$, Medium $0.35 < 0.55$, Monitor < 0.35	Thresholds are prototype configuration and must be validated in testing.
OD-10	ML role	Optional/controlled: supervised landslide susceptibility model if labeled inventory quality is adequate; otherwise deterministic susceptibility remains authoritative prototype baseline	AI must improve evidence, not be added for appearance.
OD-11	Optimization objective	Minimize weighted travel distance + unmet demand + site overload + residual hazard exposure; subject to capacity and hard safety constraints	Use OR-Tools/CP-SAT or a transparent linear/integer optimization implementation.
OD-12	Technology stack	Python + FastAPI + PostgreSQL/PostGIS + GeoPandas/Rasterio + scikit-learn/XGBoost (optional ML) + OR-Tools + React + MapLibre/Leaflet + Docker	Widely used, modular, demo-friendly, and suitable for GIS-heavy implementation.

Important: The weights and thresholds above are frozen as prototype configuration, not claimed as official government standards. They must be visible in the system's methodology/about screen and versioned in the database.

3. Data Strategy — What We Will Actually Use

The current public-source audit supports the following foundation. Official sources confirm that Chamoli has a district disaster-management plan, Census 2011 district-level village/town demographic and infrastructure data, Bhuvan disaster/GIS layers including landslides, settlements, infrastructure and hydrology, IMD daily rainfall through NWIC, and CWC telemetry river-discharge data through NWIC. These sources are the backbone of the prototype.

Domain	Preferred source	Role	Required processing
Administrative boundaries	Uttarakhand OGD / NWIC Admin Boundaries; LGD if needed	District/block/village hierarchy	Normalize names/codes; topology validation
Population	Census 2011 Chamoli PCA	Exposure + vulnerability	Join village IDs; preserve Census year in schema
Village amenities	Chamoli District Census Handbook	Water, health, education, transport, electricity and other service proxies	Normalize categorical amenities into numeric indicators
Landslide inventory/hazard	NRSC/Bhuvan + GSI resources where accessible	Training/validation + baseline hazard	Reproject; deduplicate; metadata and confidence fields
Terrain	SOI/other documented open DEM source	Elevation, slope, aspect, terrain constraints	Generate slope/aspect; preserve native resolution
Rainfall	IMD daily rainfall via NWIC; CWC telemetry where useful	Trigger and temporal update features	Quality checks; rolling rainfall features
River/discharge	CWC via NWIC; Bhuvan/India-WRIS hydrology	Flood/flash-flood exposure and update features	Station-to-raster/river association; temporal aggregation
Land cover / land use	Bhuvan/other documented open source	Suitability exclusions + score	Reclassify into allowed/conditional/excluded
Roads / infrastructure	Bhuvan + documented open geodata	Exposure + accessibility	Network-ready geometry; criticality tags
Settlements / POIs	Bhuvan + Census + documented open data	Habitation exposure and service access	Create habitation master table
Historical disasters	USDMA DDMP + NRSC/GSI/Bhuvan records where machine-readable	Recurrence/validation	Normalize event date, type, location, severity/source
Candidate relocation sites	Derived from GIS	Suitability + capacity	Generate only after hard hazard exclusions

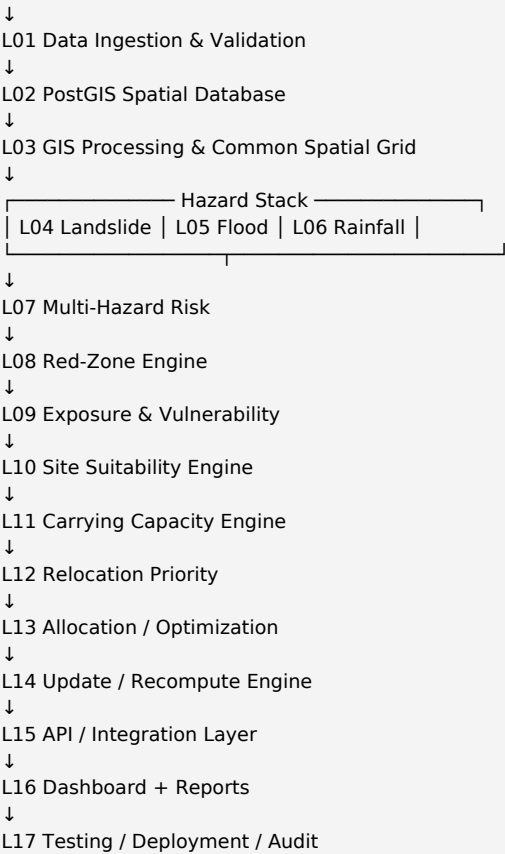
Verified source notes

- USDMA publishes a dedicated District Disaster Management Plan page that includes a Chamoli plan.
- Census 2011 Chamoli DCHB provides village/town demographic data plus infrastructure information such as education, medical facilities, drinking water, communication, transport and electricity.
- NWIC currently exposes district-wise and state-wise daily IMD rainfall CSV resources.
- NWIC currently exposes CWC Uttarakhand river-discharge telemetry datasets in CSV/API form.
- Bhuvan exposes landslide inventory/hazard resources and layers such as settlement locations, infrastructure, hydrology, land cover, POIs and river-bank erosion.

Data freshness rule: Census 2011 remains the baseline population dataset. Do not silently present it as current population. If a later population source is added, the data version and estimation method must be displayed.

4. Final System Architecture

DATA SOURCES



This preserves the original 17-layer architecture from the Master Specification while making the execution order explicit. The AI/ML component is not a replacement for the deterministic pipeline; it plugs into the hazard or vulnerability stages when data quality supports it.

System boundaries

Inside prototype	Outside prototype / explicitly not claimed
GIS hazard layers and scoring	Legal declaration of a Red Zone
Relative site suitability	Government approval of relocation sites
Modeled effective carrying capacity	Engineering certification of infrastructure capacity
Relocation prioritization	Compulsory relocation orders
Optimization scenarios	Guaranteed optimal real-world allocation
Configurable ML susceptibility model	Guaranteed hazard prediction
Updateable data pipeline	Universal real-time early-warning system

5. Red-Zone Engine — Final Prototype Definition

A Red Zone is a GIS decision layer identifying locations that fail one or more non-negotiable safety constraints or exceed the configured combined multi-hazard risk threshold. It is not a legal land-use designation.

Step 1 — Hard exclusions

- Very high landslide hazard/susceptibility class, where the authoritative layer or validated prototype threshold supports such classification.
- Flood/flash-flood exclusion areas from the selected hazard representation.
- Configurable river/active-channel buffer where the project has defensible geometry and policy justification.
- Extreme slope threshold (prototype configuration: $>45^\circ$ hard exclusion for permanent-habitation candidates; sensitivity-tested).
- Protected/restricted land-use categories where source data identifies them as unavailable for relocation.
- Known infrastructure or water-body footprints that cannot be treated as buildable land.

Step 2 — Normalized risk score

Normalize each hazard to 0–1: L = landslide risk, F = flood/flash-flood risk, R = rainfall-trigger index. Prototype combined hazard score: $H = 0.45L + 0.35F + 0.20R$. If an authoritative hazard score is available, use it as the corresponding component rather than inventing a second model.

Step 3 — Red-Zone threshold

Prototype default: Red Zone if any hard exclusion is true OR $H \geq 0.70$. Amber/High-risk: $0.55 \leq H < 0.70$. Lower-risk: $H < 0.55$. Thresholds are configuration parameters and must be sensitivity-tested; they are not official government thresholds.

Step 4 — Explanation payload

Every red-zone polygon/cell must store: triggering rule(s), L/F/R component values, combined H score, source dataset IDs, data timestamps, model/config version, and uncertainty/quality flags.

Why this design: it prevents the common mistake of equating one hazard map with 'unsafe for habitation'. It also lets the dashboard explain exactly why a location was classified.

6. Exposure & Vulnerability Engine

The unit of decision is the habitation/village or a spatially defined settlement cluster. Hazard rasters are intersected with settlement geometry to estimate exposed population and hazard intensity.

Prototype vulnerability score

$$V = 0.35P + 0.25S + 0.20A + 0.10I + 0.10D$$

Component	Meaning	Example features
P — population exposure	How many people/households are in affected area	Population, households, exposed share
S — social vulnerability	Population characteristics available from Census 2011	Age proxies, literacy, housing/amenity proxies where defensible
A — access/service deficit	Difficulty reaching essential services	Road access, healthcare, education, drinking-water availability
I — infrastructure dependency	Dependence on vulnerable infrastructure	Road/bridge/service exposure
D — disaster recurrence	Historical event recurrence/exposure	Normalized count or recurrence index from sourced events

All component scores are normalized to 0-1 before weighting. Missing data must not silently become zero; missingness is represented separately and can trigger a confidence penalty.

Exposure calculation

For each habitation h : $\text{exposed_population} = \text{population}(h) \times \text{affected_fraction}(h)$, where affected_fraction is derived from spatial intersection with the relevant hazard layer(s). If exact building-level population is unavailable, village/settlement-level allocation is used and clearly labeled as an approximation.

7. Relocation-Site Suitability Engine

Candidate sites are generated algorithmically; they are not hand-picked as if they were already approved government relocation sites.

Candidate generation pipeline

Chamoli boundary → remove Red Zone → remove restricted land → remove water bodies → apply slope constraint → apply minimum contiguous area → retain cells/polygons with road access → cluster adjacent eligible cells → generate candidate site polygons → score and rank.

Suitability score

$$S = 0.30H_{\text{safe}} + 0.15S_{\text{slope}} + 0.15R_{\text{oad}} + 0.15W_{\text{ater}} + 0.10H_{\text{ealth}} + 0.05E_{\text{ducation}} + 0.05L_{\text{andUse}} + 0.05S_{\text{ervices}}$$

Here H_{safe} is the inverse of modeled hazard, after hard exclusions. Every factor is normalized to 0–1 where 1 is more suitable.

Minimum candidate-site rules

- No candidate site may overlap a hard Red Zone.
- Candidate must have a contiguous area above configurable minimum area.
- Candidate must satisfy the prototype slope rule or be explicitly marked conditional.
- Candidate must have measurable access to an existing road network or be marked 'new-access required'.
- Water/health/education constraints must be represented as either direct capacity data or clearly labeled proxy indicators.
- The dashboard must show why a site scored high/low.

8. Carrying Capacity Engine

The project reports modeled effective carrying capacity, not an engineering-certified population capacity.

Capacity components

Component	Prototype calculation	Output
Land capacity	$\text{usable_area} \times \text{target_density}$	people
Water capacity	$\text{available/estimated daily water capacity} \div \text{per-capita requirement}$	people
Sanitation capacity	estimated sanitation service capacity or conservative proxy	people
Healthcare capacity	$\text{service capacity} / \text{accessibility proxy}$	people
Access capacity	road/access bottleneck proxy	people
Safety capacity	hazard-safety factor applied to feasible capacity	people

Default bottleneck rule: $\text{physical_capacity} = \min(\text{land, water, sanitation, healthcare, access})$. Then $\text{effective_capacity} = \text{floor}(\text{physical_capacity} \times \text{safety_factor})$. Prototype default $\text{safety_factor} = 0.80$ unless a source-backed engineering constraint is available.

The system must expose every component so a reviewer can see the bottleneck. Example: if land allows 1,200 people but water supports only 750, the effective capacity cannot be reported as 1,200.

No fabricated infrastructure capacity: if direct capacity is unavailable, use a proxy and label it as proxy. The UI must distinguish measured/source-backed values from modeled estimates.

9. Relocation Priority & Allocation

Priority score

$$RP = 0.40Risk + 0.25ExposedPopulation + 0.20Vulnerability + 0.10ResponseDifficulty + 0.05Recurrence$$

Tier	Score	Interpretation
Immediate	≥ 0.75	Highest modeled relocation urgency; requires authority review
Short-term	$0.55 - < 0.75$	Needs planned action in the near planning horizon
Medium-term	$0.35 - < 0.55$	Planned mitigation/relocation candidate
Monitor	< 0.35	Lower modeled priority under current inputs

Allocation optimization

Decision variables: $x[h,s]$ = number of people allocated from habitation h to site s . Objective: minimize weighted travel distance + unmet population penalty + site overload penalty + residual hazard exposure.

Core constraints: $x[h,s] \geq 0$; $\sum_s x[h,s] \leq \text{exposed_population}(h)$; $\sum_h x[h,s] \leq \text{effective_capacity}(s)$; $x[h,s] = 0$ when site s is infeasible for h ; optionally enforce maximum travel distance.

Implementation: OR-Tools CP-SAT or MILP. Keep the objective and constraints readable in code; do not hide them inside an opaque AI model.

10. AI / ML Strategy — What AI Actually Does

AI is not the whole product. The strongest implementation is a hybrid system: deterministic GIS for safety constraints, ML for a narrowly defined prediction task when training labels support it, and optimization for allocation.

Primary optional ML task: landslide susceptibility

- Training labels: sourced landslide inventory points/polygons with explicit source/date/quality.
- Features: slope, aspect, elevation, rainfall-derived features, land cover, geology/soil where available, distance to drainage, distance to roads, curvature/terrain derivatives where available.
- Candidate models: Random Forest first; XGBoost as a controlled comparison; logistic regression as an interpretable baseline.
- Split by geography or spatial blocks where possible to reduce spatial leakage.
- Metrics: ROC-AUC/PR-AUC plus calibration, confusion matrix at the operational threshold, and spatial sanity checks.
- Output: probability/susceptibility raster + uncertainty/quality metadata.
- Do not use ML if labels are too sparse, biased, or inaccessible; retain deterministic hazard baseline.

AI-assisted engineering workflow

Coding AI should be used as a controlled implementation assistant, not as the architect. It receives the relevant layer specification, repository conventions, schemas, acceptance tests and constraints, then produces code only for that layer.

LLM features inside the product

- Optional natural-language explanation of a result using structured system outputs.
- Report drafting from already-computed facts.
- Query-to-filter assistance for the dashboard, if time permits.
- Never allow an LLM to directly change hazard scores, Red Zone boundaries, capacities, or allocation decisions.

11. Layer-by-Layer AI Development Plan

Phase	Layer	Build output	AI prompt input	Definition of done
1	L01 Data	Download/import/validation scripts + raw/staging/curated folders	L01 spec + source manifest	All selected datasets ingest; provenance recorded; validation report passes
2	L02 Spatial DB	PostGIS schema, indexes, geometry constraints	DB schema + naming rules	Queries and spatial joins work on clean sample
3	L03 GIS	Common CRS/grid + terrain derivatives + spatial utilities	GIS processing contract	Reproducible raster/vector outputs generated
4	L04 Hazard	Landslide baseline	Hazard spec + source layers	Hazard layer produced with metadata and tests
5	L05 Flood	Flood/flash-flood baseline	Flood spec + river/rainfall data	Flood exposure layer + validation checks
6	L06 Rainfall	Trigger index + temporal features	Rainfall spec	Daily/rolling metrics generated correctly
7	L07 Risk	Multi-hazard normalized score	Risk formulas + configs	Risk raster/cells reproducible
8	L08 Red Zone	Hard constraints + H score	Red-zone rules	Red/amber/other classes with explanations
9	L09 Vulnerability	Habitation exposure/vulnerability	Vulnerability formulas	Rankable habitation table
10	L10 Suitability	Candidate site generation + scoring	Site rules	Candidate sites generated without hard-zone overlap
11	L11 Capacity	Effective capacity model	Capacity formula + proxies	Capacity components visible
12	L12 Priority	Immediate/short/medium tiers	Priority formula	Priority reproducible from DB
13	L13 Optimization	Habitation→site allocation	Objective + constraints	Feasible allocations produced and audited
14	L14 Updates	Recompute/update jobs	Data freshness rules	Changed inputs trigger deterministic recomputation
15	L15 API	FastAPI endpoints	API contract	Frontend can consume every required output
16	L16 Dashboard	Map + analytics + explanations	UI spec	End-to-end user workflow works
17	L17 QA/Deploy	Tests + Docker + demo dataset	Acceptance matrix	Clean setup, repeatable demo, no critical test failures

Rule: Do not ask the coding AI to build all 17 layers in one prompt. One layer = one implementation contract = one testable merge/checkpoint.

12. Repository Structure — Freeze This Before Coding

```
sih-26191/ ├── docs/ | ├── master-blueprint-v1.0.pdf | ├── layer-specs/ | └── data-dictionary.md ├── data/ | ├── raw/ | ├── staging/ | ├── curated/ | └── derived/ ├── config/ | ├── hazards.yaml | ├── weights.yaml | ├── thresholds.yaml | └── sources.yaml ├── db/ | ├── migrations/ | └── seeds/ ├── src/ | ├── ingestion/ | ├── validation/ | ├── spatial/ | ├── hazards/ | ├── risk/ | ├── vulnerability/ | ├── suitability/ | ├── capacity/ | ├── priority/ | ├── optimization/ | ├── api/ | ├── common/ | ├── ml/ | ├── features/ | ├── training/ | ├── evaluation/ | ├── artifacts/ ├── frontend/ | ├── map/ | ├── dashboard/ | ├── components/ ├── tests/ | ├── unit/ | ├── spatial/ | ├── integration/ | ├── acceptance/ ├── notebooks/ ├── scripts/ ├── docker/ ├── docker-compose.yml └── README.md
```

Naming rules

- Python: snake_case; classes: PascalCase; API routes: /api/v1/...
- Database tables: singular logical domain names or a consistent plural convention; do not mix conventions.
- Every spatial table includes geometry SRID, source_id, dataset_version, created_at, updated_at where applicable.
- Every derived result includes model_version/config_version.
- Never overwrite raw data. New source versions create new raw records.

13. Core Database Schema

Table	Purpose	Key fields
data_source	Provenance registry	source_id, agency, url, access_date, license, version, checksum
admin_unit	Administrative hierarchy	unit_id, type, parent_id, name, code, geom
habitation	Decision unit	habitation_id, admin_unit_id, name, population_year, population, households, geom
infrastructure	Critical assets	asset_id, type, name, capacity/proxy, geom, source_id
hazard_layer	Hazard metadata	hazard_id, type, date/version, source_id, model_version, raster/vector reference
risk_cell	Canonical grid outputs	cell_id, h_landslide, h_flood, h_rain, combined_risk, red_zone, quality_flag
vulnerability	Habitation scores	habitation_id, population_exposure, social_score, access_score, infra_score, recurrence_score, vulnerability
candidate_site	Relocation candidates	site_id, geom, area, suitability, status, explanation_json
capacity	Site capacity	site_id, land_cap, water_cap, sanitation_cap, health_cap, access_cap, effective_cap
priority	Relocation urgency	habitation_id, risk, exposed_pop, vulnerability, response_difficulty, recurrence, priority_score, tier
allocation	Optimization result	run_id, habitation_id, site_id, allocated_population, distance, constraint_flags
model_run	Reproducibility	run_id, model_version, config_version, input_versions, started_at, status

Spatial reference rule: choose one projected CRS appropriate for distance/area calculations in the study region; use WGS84/lat-long only for interchange/API display. The exact EPSG code must be stored in config and documented in L03 rather than scattered through code.

14. API Contract

Endpoint	Purpose
GET /api/v1/admin-units	Administrative boundaries
GET /api/v1/habitations	Habitation list/filter
GET /api/v1/habitations/{id}	Habitation profile + risk/vulnerability explanation
GET /api/v1/hazards	Available hazard layers/metadata
GET /api/v1/risk/map	Map-ready risk/red-zone tiles or GeoJSON for small AOI
GET /api/v1/sites	Candidate relocation sites with suitability
GET /api/v1/sites/{id}/capacity	Capacity components + effective capacity
GET /api/v1/priorities	Immediate/short/medium/monitor list
POST /api/v1/optimize	Run allocation scenario from validated inputs
GET /api/v1/runs/{id}	Model/optimization run status + provenance
GET /api/v1/reports/{id}	Download generated decision report

API rule: The frontend never recomputes authoritative scores. It requests results from the backend. This prevents different UI components from silently using different formulas.

15. Dashboard — Final Screens

Screen	Must show
1. Overview	Study area, current data version, key metrics, last successful processing run
2. Hazard Map	Landslide, flood, rainfall layers; legend; source/date
3. Red Zone Map	Red/amber/other, click explanation, contributing hazards
4. Habitation Risk	Search/filter habitations, population, exposed population, vulnerability, priority tier
5. Site Explorer	Candidate sites, suitability score, constraints, nearby services
6. Capacity	Site capacity components, bottleneck, effective capacity
7. Relocation Planner	Select habitation(s), candidate sites, scenario constraints
8. Allocation Result	Assigned population, travel distance, unmet demand, site load
9. Methodology	Weights, thresholds, data sources, model versions, limitations
10. Reports	Generate/export decision-support summary

Map interaction rule: every colored layer must have a legend, unit, timestamp/version, and source. Never show a color ramp without explaining what the numeric value means.

16. Testing & Acceptance Criteria

Every layer must have unit tests, data validation tests, and at least one end-to-end acceptance test.

Test group	Examples
Data tests	Required columns, null rate, valid geometries, CRS, duplicate IDs, source checksum/version
Spatial tests	No invalid polygons, expected overlap behavior, area sanity checks, raster bounds match study area
Hazard tests	Known event areas receive non-zero hazard; no-data areas remain flagged
Red-zone tests	Hard exclusion always overrides low score; threshold classification reproducible
Vulnerability tests	Changing population changes exposure/priority predictably; missing data never silently becomes zero
Suitability tests	Candidate cannot overlap hard Red Zone; excluded land cannot become eligible
Capacity tests	Effective capacity never exceeds every binding bottleneck
Optimization tests	No allocation exceeds site capacity; infeasible assignments are zero/flagged
API tests	Schema validation, error codes, pagination/filter behavior
UI tests	Map layer toggles, click explanations, scenario submission, report generation
Reproducibility	Same input versions + same config produce same deterministic outputs

Minimum demo acceptance: starting from a clean environment, a reviewer can ingest the supplied demo data, run the pipeline, open the map, click a Red Zone, inspect a habitation, inspect a relocation site, see its capacity bottleneck, run an allocation scenario, and export a report.

17. How You Should Use Coding AI — Exact Workflow

You do not need to repeatedly explain the entire project to the AI. Keep the blueprint in the repository and give the coding AI the relevant layer specification plus the repository state.

For every layer, use this 7-step loop:

- Give AI the master blueprint and the specific layer contract.
- Give the current repository tree and the files it is allowed to modify.
- Give the input/output schemas and acceptance tests.
- Ask AI to first inspect the existing code and list conflicts; do not let it redesign architecture.
- Ask AI to implement only that layer.
- Run tests and a small real-data smoke test.
- Only after acceptance passes, move to the next layer.

Reusable master prompt for every coding AI

You are implementing Layer {LAYER_ID} of SIH Problem Statement 26191. Treat docs/master-blueprint-v1.0.pdf as the source of truth. Do not redesign architecture, change frozen decisions, invent data, invent APIs, or add dependencies without explicit justification. First inspect the repository and identify existing interfaces that this layer must respect. Then implement only this layer. Use the exact input/output contracts and configuration keys defined in the layer specification. Write production-quality modular code with logging, validation, error handling and tests. Never hard-code secrets or source URLs inside business logic. Every derived result must carry source/version/model/config metadata where applicable. After implementation, run the relevant tests and provide: 1) files changed, 2) schema/API changes, 3) commands run, 4) test results, 5) known limitations, 6) exact next dependency.

If AI starts proposing a new architecture, stop it. Tell it: “Architecture is frozen. Implement within the existing contract; do not redesign.”

18. Ready-to-Use AI Prompts by Phase

L01 — Data Layer prompt

Implement L01 only. Build source manifests, download/import adapters, raw/staging/curated structure, schema validation, CRS checks, geometry checks, null/duplicate reports, dataset versioning and provenance. Do not build hazard scoring, UI, ML or optimization. Produce tests and a sample data-quality report.

L02 — PostGIS prompt

Implement L02 only. Create migrations for the frozen schema, spatial indexes, geometry constraints, foreign keys, version/provenance columns and seed/config tables. Do not alter formulas or frontend behavior. Add database tests.

L03 — GIS prompt

Implement L03 only. Establish canonical projected CRS/config, 30 m analysis grid, raster/vector reprojection utilities, clipping, slope/aspect derivation, distance-to-feature utilities and safe resampling rules. Preserve native source resolution metadata.

L04-L06 — Hazard prompts

Implement only the named hazard module. Start from source-backed baseline layers. Normalize output to 0–1 only after documenting the normalization method. Keep raw hazard output and normalized output separate. Add validation against known historical events where possible.

L07-L09 — Risk/Red Zone/Vulnerability prompt

Implement the frozen formulas exactly from config. Hard constraints must be evaluated before soft scoring. Produce explanation_json for every habitation/cell where practical. Never silently fill missing data with zero.

L10-L13 — Site/Capacity/Priority/Optimization prompt

Implement candidate generation, suitability, capacity and optimization exactly as specified. Every site must expose component scores and bottlenecks. Every allocation must expose source habitation, destination site, allocated population, distance and constraint flags.

L14-L17 — Update/API/UI/QA prompt

Connect existing deterministic outputs without duplicating formulas in the frontend. Add version-aware endpoints, map rendering, explanations, scenario execution, report generation, integration tests and Docker deployment. Do not add new business logic to the UI.

19. Practical AI Toolchain

Task	Use AI for	Human responsibility
Architecture	Review consistency, identify missing contracts	Approve frozen design
Coding	Generate/refactor modules, tests, migrations	Review diffs and run tests
GIS	Write GeoPandas/Rasterio/PostGIS pipelines	Verify CRS, geometry and visual outputs
ML	Feature engineering, baseline models, evaluation scripts	Verify leakage, labels, metrics and scientific validity
Optimization	Model constraints/objective, test scenarios	Verify assumptions and feasibility
Frontend	Components, map integration, API wiring	Verify UX and explainability
Debugging	Trace errors, propose fixes	Confirm root cause and regression test
Documentation	Generate API docs and technical notes	Verify against actual implementation

Recommended division of labor: use one coding AI as the primary implementer and, when available, a second AI as a reviewer/test writer. Do not let two AIs independently redesign the same layer.

Human checkpoints

- After L01: verify actual datasets can be downloaded/loaded.
- After L03: visually inspect terrain and boundaries.
- After L04–L06: compare hazard outputs with known events.
- After L08: inspect Red Zone polygons manually.
- After L10–L11: inspect candidate sites and capacity bottlenecks.
- After L13: inspect allocation sanity.
- Before demo: run full clean-install pipeline.

20. SIH Demo Strategy — What to Show

Do not attempt to demonstrate every possible hazard. Demonstrate one complete decision chain extremely clearly.

Recommended 5-minute demo flow

- Open Chamoli map and show data provenance/version.
- Turn on landslide + flood + rainfall layers.
- Show Red Zone and click a high-risk habitation.
- Display population, exposure, vulnerability and relocation priority with explanations.
- Open candidate relocation sites and compare suitability.
- Open one site and show capacity bottleneck.
- Run allocation scenario and show who goes where, how many people, and remaining unmet demand.
- Generate a report summarizing the decision and all model/config versions.
- Show methodology screen to prove that weights/thresholds are transparent and configurable.

What not to say in the pitch

- “Our AI predicts disasters with 100% accuracy.”
- “These are government-approved Red Zones.”
- “This site can legally accommodate 800 people.”
- “Our system is real-time” unless the live update mechanism is actually operating.
- “AI decided the relocation.”

What to say instead

- “The prototype produces an evidence-backed, configurable decision-support layer.”
- “Red Zone is a modeled prototype classification for authority review.”
- “Capacity is a modeled effective capacity based on explicit bottlenecks.”
- “The architecture supports updates from available government data sources.”
- “AI is used selectively where labeled data and validation justify it.”

21. Major Risks and Mitigations

Risk	Impact	Mitigation
Population data is 2011	Current exposure may be understated/overstated	Label year; allow scenario scaling; never call it current census
Hazard layers have different resolutions	False precision	Retain native resolution; canonical grid only for controlled analysis
Landslide inventory bias	ML may learn reporting/satellite bias	Spatial split, source/date fields, baseline comparison, uncertainty flag
Insufficient site-capacity data	Capacity could look fabricated	Use bottleneck proxies and label them; never invent measured capacity
Thresholds are not official	Potential overclaim	Mark all prototype thresholds/configs; sensitivity testing
Government portal changes	Pipeline break	Source manifest + adapters + cached raw versions
Too much scope	Incomplete demo	Freeze V1 to three hazard inputs and one district
AI-generated code errors	Silent technical defects	Tests, code review, smoke tests, acceptance gates
Frontend duplicates formulas	Inconsistent results	Backend is source of truth for decision outputs

22. Execution Order — Your Next 14 Working Sessions

Session	Goal	Deliverable
1	Repository + L01 spec	Repo skeleton, configs, source manifest
2	L01 ingestion	First clean datasets in raw/staging/curated
3	L01 validation	Automated quality report + provenance
4	L02 PostGIS	Migrations + spatial schema
5	L03 GIS	CRS/grid + terrain processing
6	L04 landslide	Baseline landslide layer
7	L05 flood	Flood/flash-flood baseline
8	L06 rainfall	Trigger features
9	L07-L08	Risk + Red Zone
10	L09	Exposure/vulnerability + priority
11	L10	Candidate sites + suitability
12	L11	Capacity model
13	L13-L15	Optimization + API
14	L16-L17	Dashboard + tests + Docker + demo

If a session fails its acceptance criteria, do not move forward. Fix the current layer first. This is the main rule that prevents AI-generated technical debt from propagating across the project.

23. Final Freeze Checklist

- Geography = Chamoli District, Uttarakhand.
- Core hazard inputs = landslide, flood/flash flood, heavy/extreme rainfall trigger.
- Canonical analysis grid = 30 m; native source resolution preserved.
- Red Zone = hard exclusions + $H = 0.45L + 0.35F + 0.20R$; prototype red threshold 0.70.
- Vulnerability = 35/25/20/10/10 weighting.
- Site suitability = 30/15/15/15/10/5/5/5 weighting.
- Capacity = bottleneck-based, then safety factor; modeled not certified.
- Priority = 40/25/20/10/5; tiers Immediate/Short/Medium/Monitor.
- Allocation = constrained optimization, not an LLM decision.
- ML = optional landslide susceptibility only if data quality supports it.
- PostGIS + FastAPI + Python GIS + React/MapLibre/Leaflet + OR-Tools.
- Every output has provenance/model/config version.
- Every layer has tests and acceptance criteria.
- No legal, certification, guaranteed-prediction or unsupported real-time claims.

This document now serves as the frozen implementation contract. Future layer specifications should be generated from it, not reinvented.

24. Verified Source Register

The following sources were checked during preparation. The PDF records them so the implementation team can revisit the original source rather than relying on copied summaries.

Source	Use in project
USDMA — District Disaster Management Plan: https://usdma.uk.gov.in/document-category/district-disaster-management-plan/	Chamoli DDMP and district disaster context
Census India — Chamoli DCHB Part A: https://censusindia.gov.in/nada/index.php/catalog/1306	Village/town directory and infrastructure amenities
Census India — Chamoli DCHB Part B: https://censusindia.gov.in/nada/index.php/catalog/1307/study-description	Primary Census Abstract / population
NWIC — IMD Daily Rainfall: https://nwdp.nwic.gov.in/dataset/rainfall-daily-imd	Daily district/state rainfall CSV resources
NWIC — CWC Hourly River Discharge: https://nwdp.nwic.gov.in/dataset/river-discharge-telemetry-hourly-central-water-commission-cwc	Uttarakhand river discharge telemetry
Bhuvan/NRSC — Disaster Services: https://bhuvan-app1.nrsc.gov.in/disaster/disaster.php	Disaster GIS layers and services
Bhuvan/NRSC — Landslide: https://bhuvan-app1.nrsc.gov.in/disaster/usrtasks/landslide/landslide.php?uname=empty	Landslide inventory/hazard resources
Bhuvan/NRSC — Uttarakhand 2013 landslide study: https://bhuvan-app1.nrsc.gov.in/disaster/usrtasks/landslide/doc/Uttarakhand_Landslide_inventory_2013.pdf	Historical landslide inventory and methodological reference
Uttarakhand OGD — Admin Boundaries: https://uttarakhand.data.gov.in/catalog/admin-boundaries	State/district/block boundaries
Bhuvan/NRSC — Thematic Dashboard: https://bhuvan-app1.nrsc.gov.in/thematic_dashboard/	Thematic GIS layer access and operations
Bhuvan/NRSC — Uttarakhand Forest viewer: https://bhuvan-app1.nrsc.gov.in/uk_forest/	Uttarakhand GIS layers including land cover/landslides/settlements

Source limitations: Public portal availability, formats, licensing, access controls and update schedules can change. The implementation must record the actual access date, dataset version and license/usage note for every imported dataset.

End of Final Blueprint v1.0